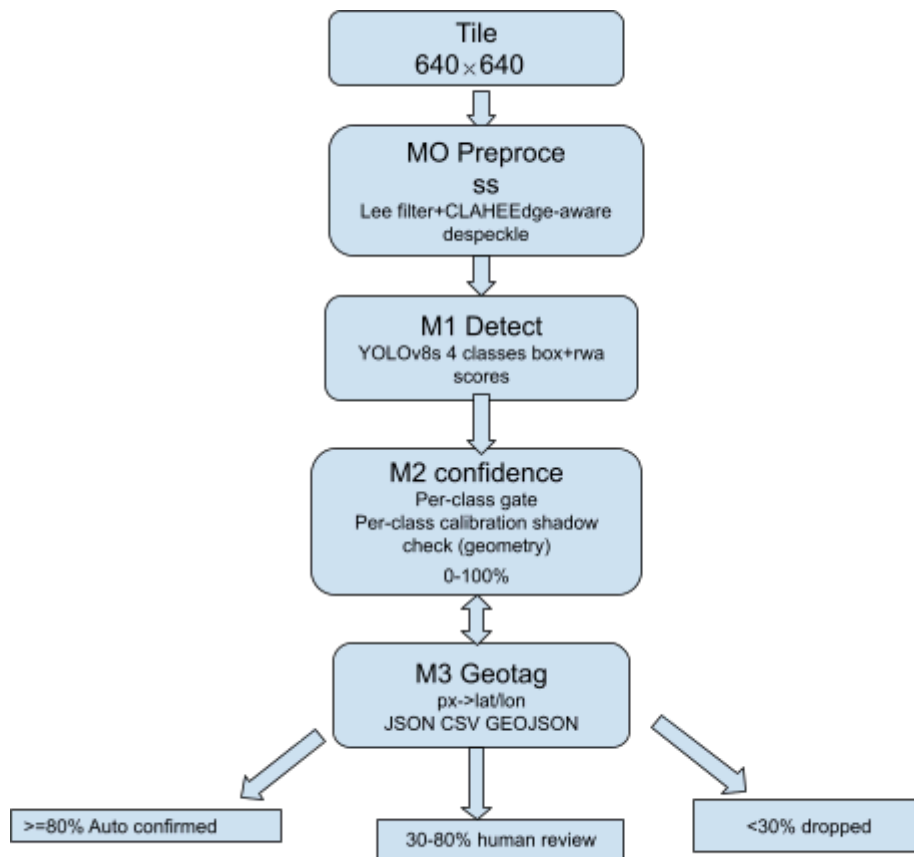


1. Overall System Workflow



In simple words, the system first receives sonar data. It improves the sonar image and sends it to the AI model. The model finds possible objects. The system then checks how reliable the detection is. After that, it finds where the object is located and stores the result.

Module 0 – Preprocessing

The first module prepares the sonar image before it is given to the AI model. Side-Scan Sonar images are different from normal photographs. They can contain speckle noise, uneven brightness and strong dark or bright regions. Acoustic shadows are also important because they can help identify objects.

For the current system, two main image processing operations are used:

Lee Filter:-The Lee filter is used to reduce speckle noise while trying to keep important edges. This is useful because removing too much noise can also remove small parts of an object. The basic idea is: Raw sonar image → Lee filter → cleaner sonar image

The Lee filter used in the project uses a 7×7 window.

CLAHE:-CLAHE means Contrast Limited Adaptive Histogram Equalization. It improves local contrast in the sonar image. This can make some objects and their acoustic shadows easier to see. The important point is that the same preprocessing is used during training and during inference. This prevents the model from seeing one type of image during training and a very different type during testing. The current preprocessing flow is therefore:

Raw SSS → Lee Filter → CLAHE → 640×640 image → YOLOv8s

The other information can be used later when the system receives complete sonar and navigation data.

Module 1 – Object Detection

The second module is the main AI detection part.DRISHTI uses **YOLOv8s** for object detection.The model takes a 640 × 640 sonar image and tries to find objects inside it.Why 640 × 640? A 640 × 640 input provides a practical balance between retaining small and fragmented sonar structures and keeping inference computationally lightweight. Larger inputs may preserve more fine detail but increase memory use and inference time, while smaller inputs may lose thin or small targets.Unlike simple image classification, object detection gives more information.For every detected object, the model provides:Object class,Bounding box,Raw confidence score. For example, the model may find a possible shipwreck and return a bounding box around it.This information is important because the location of the bounding box can later be converted into geographical coordinates.The selected YOLOv8s model has about **11.8 million parameters**.The final held-out test result was:mAP@50: **0.641**,precision: **0.734**,recall: **0.629**. The pipeline class performed very strongly, while shipwreck and mine-cylinder detection were more difficult.

Module 2 – Confidence and Verification

The AI model gives a score for every detection. However, this score should not automatically be treated as a true probability.A high model score does not always mean that the detection is correct.Therefore, DRISHTI uses an additional confidence module. This module performs:Per-class confidence gates,per-class calibration,acoustic shadow and geometry checking.Acoustic shadow and geometry verification is intended to operate on the original sonar transect together with the available sonar and navigation metadata. An isolated 640 × 640 image tile does not contain all of the information required for complete shadow-length or geometric verification. Therefore, this verification stage should be treated as a full-transect processing capability rather than a property that can always be evaluated from an individual image tile. The calibration step makes the confidence values more meaningful. The Expected Calibration Error (ECE) improved from:0.052 → 0.037.This means the confidence values became better calibrated.

Calibration Parameters

The confidence calibration uses class-specific Platt scaling parameters. The calibration coefficients used by the implementation are recorded below:

Class	a	b
Pipeline	4.20	−0.82
Shipwreck	5.22	−2.22
Ghost Net	4.93	−1.60
Mine Cylinder	5.20	−2.35

These parameters are used to transform the detector's raw confidence scores into calibrated confidence values before applying the class-specific decision thresholds.

The system also uses different thresholds for different classes because the classes do not have the same detection difficulty.For example, a pipeline is usually easier to identify than a faint ghost net.

Per-Class Confidence Gates

The confidence module applies class-specific confidence gates because the target classes have different detection characteristics. The thresholds used by the current system are:

Class	Confidence Threshold
Pipeline	0.25
Shipwreck	0.40
Mine Cylinder	0.40
crab_pot	0.99
Ghost Net	0.30

These values should be kept consistent with the thresholds used by the deployed inference and verification pipeline.

Module 3 – Geotagging

After detecting an object, we need to know where the object is located. This is the job of Module 3. The system converts the image position into latitude and longitude. The basic process is: Image row → Ping number → Navigation record and Image column → Slant range → Ground range

The system uses sonar ping information and navigation information from the vehicle.

For the ground-range calculation, the system uses: $G = \sqrt{R^2 - H^2}$

where: G is ground range, R is slant range and H is vehicle or sonar height.

After finding the ground range, the system uses the vehicle's position and heading to estimate the target's geographical position. The result can include: Latitude, Longitude, Object size, Detection confidence, Object class and Ping ID. The system can then save this information in formats such as: JSON, CSV and GeoJSON

The navigation position and the detection position are not exactly the same thing. The tow-fish navigation position was validated to less than 1 metre against an independent navigation source. However, the final detection location also depends on the sonar image scale and across-track position. During testing, two navigation paths placed the same target about 122 metres apart. The 122-metre difference is an observed discrepancy between two navigation-based placement results and should not be interpreted as a validated 122-metre geolocation error. The underlying cause requires further investigation, which may include effects from tow-fish trajectory, across-track estimation, sonar geometry, or synchronization between sonar and navigation data.

Because of this, DRISHTI should not show every detection as if it has exact metre-level accuracy. Instead, the system should show an uncertainty area or uncertainty circle around the detection. This is an important part of the system because it prevents the dashboard from giving a false impression of location accuracy.

2. Edge Deployment

One of the important requirements of Problem Statement 26057 is offline operation. An underwater vehicle cannot depend on a normal internet connection. Because of this, DRISHTI uses an ONNX-based deployment path. The deployed system uses: ONNX Runtime, NumPy and OpenCV. PyTorch is not required during the final inference process. This makes the runtime much smaller. The measured installed

runtime was approximately:210 MB compared with about:4.4 GB when the PyTorch environment is installed.The actual model file is also small enough for edge deployment.

ONNX Model Testing

Different ONNX precisions were tested.

Version	Size	CPU Result	Decision
FP32 ONNX	44.8 MB	~90 ms/tile	Used
FP16 ONNX	22.4 MB	Not tested on CPU	Jetson target
INT8 ONNX	11.5 MB	~164 ms/tile	Not used

The INT8 model was not selected.Although it was smaller, its measured precision and recall became approximately zero for the tested data.It was also slower than the FP32 version on the tested CPU.Therefore, DRISHTI uses the working FP32 ONNX version for the current deployment path.FP16 with TensorRT is planned as a target for the Jetson GPU deployment.

The internal training dataset contains 6,405 tiles. The publicly distributable release contains 5,205 tiles because the crab-pot class was excluded due to licensing restrictions. Therefore, the internal experimental dataset and public release should not be treated as the same dataset.

3. Target Hardware

The reference underwater platform for the system is the L3Harris Iver4 900 AUV.The sonar reference is the EdgeTech 2205.For onboard AI processing, the project recommends a:NVIDIA Jetson Orin NX 16 GB The Jetson provides a suitable platform for running the AI model locally.The important point is that the vehicle does not need to send the full sonar image to a remote server for every detection.The AI can process the data on the vehicle itself.This is important for underwater operation.An underwater vehicle has a major communication limitation.An acoustic modem cannot send full sonar images at a useful speed.Therefore, sending the complete sonar image to a shore-based AI model is not practical during the underwater mission.DRISHTI follows a different approach.

The vehicle:Receives sonar data,processes the data locally,runs the detection model locally,stores the results locally,sends only small contact information when communication is available.
A small contact summary can contain:Class,position,confidence,radius,ping ID.The complete sonar data can be transferred after the vehicle returns.This is one of the main reasons for using edge AI.

4. Three Operating Modes

The system is designed around three operating situations.

Mode A – Demo Mode:-In the demo mode, a user uploads sonar data through a browser.The flow is:Browser → Django API → Celery Worker → AI Model → Database → Dashboard
The worker processes the uploaded data and sends the results back to the dashboard.WebSocket communication can be used to show results progressively.This mode is useful for demonstrations and testing.

Model Integration Architecture

The AI inference pipeline is integrated into the Celery worker as an in-process model library rather than as a separate ML microservice. The worker can import and call the inference pipeline directly, allowing sonar processing and detection to run without sending model inputs through an additional network

service. This reduces network serialization overhead and avoids introducing a separate inference-service failure point into the processing pipeline.

Mode B:- Live AUV Mode:-In the live AUV mode, the sonar is connected directly to the onboard computer. The flow is: Sonar → Jetson Orin NX → Local Storage
The AI model runs on the vehicle. The detection results are stored locally during the mission. Only small contact summaries may be sent through an acoustic communication line. After recovery, the stored data can be transferred using a faster connection such as Ethernet or removable storage.

Mode C – Public Debris Map:-The third mode is a possible public-facing application. The idea is to show confirmed underwater hazards to people who may need this information. However, only analyst-confirmed detections should be shown publicly. The map should also use uncertainty circles instead of pretending that every detection has exact coordinates. There are also safety concerns. For example, shipwrecks may be protected heritage sites. Possible mine or ordnance detections should also not simply be published as normal map locations. Therefore, this mode requires proper authority and safety controls before public deployment.

5. Technology Stack

The main technologies used in the system are:

AI and Inference:-YOLOv8s, ONNX Runtime, NumPy and OpenCV

Backend:-Django REST, Django Channels, Celery and Redis

Database:-PostgreSQL

Mapping:-React and React-Leaflet

Geospatial Processing:-pyproj and Shapely

The project was originally pointed toward FastAPI and GeoAlchemy2 for some parts of the architecture. However, the actual system uses Django REST and the Django ORM.

Why Django is Used

FastAPI is a good option for lightweight APIs. However, the actual edge inference process does not run a web server. The vehicle uses the ONNX inference program directly. The server-side part of DRISHTI needs more than a simple API. It needs: Background jobs, WebSocket updates, user authentication, review queues, database management and Job tracking.

Django with Channels and Celery fits these requirements. Celery is used for longer-running background tasks, while Redis is used as the task broker. This makes the server side suitable for processing a complete sonar transect without blocking the user interface.

6. Offline Capability of Each Part

The system is designed so that the important underwater processing does not require the internet.

System Part	Offline?	Explanation
AI inference	Yes	ONNX Runtime runs locally
Preprocessing	Yes	Uses local sonar data
Geotagging	Yes	Uses local navigation information
Storage	Yes	Results stored on vehicle
Map	Yes	Local cached map data can be used
Dashboard	No on vehicle	Mainly surface-side after recovery

The most important part is the inference pipeline. It should continue working even when there is no network connection.

7. Human Review Is Still Important

DRISHTI is designed to reduce the amount of manual work, but it does not assume that every AI prediction is correct. This is important because sonar images can be difficult to interpret. A bright seabed region, rock, fishing equipment or another object can sometimes look similar to a target. Therefore, the system uses a human review step for uncertain detections. The basic approach is: **High confidence** → **automatic confirmation**, **Medium confidence** → **human review**, **Low confidence** → **remove**

This gives the analyst a smaller and more useful list to inspect instead of asking them to manually check the entire sonar survey.

8. Current System Status

The major system components have different levels of completion.

Module	Current Status
Preprocessing	Live
Object Detection	Shipped
Confidence Calibration	Functional
Geotagging	Completed
Dashboard	Wiring / scaffold stage
Edge ONNX inference	Working
Public debris map	Future/controlled mode

This distinction is important. The AI detection and geotagging parts have been tested, while some user-interface and public-facing features still need further development.